

PRD 1: Grounded Document Q&A over Insurance Policy Documents

Project 1 of 5 · Python · Phase 1, weeks 3 to 7 · Suggested repository name: `grounded-docs` · Version 1.0, 30 September 2026

1. Summary

A question-answering service over a corpus of real insurance policy wordings and regulator guidance. It answers only from the documents, cites page-level sources, refuses when the evidence is insufficient, and publishes measured retrieval and answer quality, including an ablation table that shows what each pipeline stage contributes. The retrieval API is reused by Project 2 (the claims agent) and Project 4 (the MCP server).

2. Why this project

Evidence from the September 2026 market study	Implication
RAG, embeddings or vector search appear in 19 of 27 full postings	Every candidate needs one; yours must be visibly beyond the baseline
Document Q&A is over 40 percent of published take-home assignments	Building this once is take-home rehearsal
"Design enterprise document Q&A" is the most common LLM system design question	You will defend this architecture in interviews
Postings ask for "diagnosing retrieval quality" and "RAG experience beyond basic chatbots"	Retrieval metrics and an ablation table are the differentiator
A candidate quoting "hybrid BM25 + dense retrieval and cross-encoder reranking, Hit@5 1.00, MRR 0.94" drew responses in the September 2026 Hacker News thread	Publish the same class of numbers
Thoughtworks Radar 34 puts context engineering on Adopt; Chroma's research shows long-context degradation with distractors	"Just stuff the context window" is not an acceptable answer

3. Problem statement

Policy servicing agents, claims handlers and compliance analysts need precise answers to questions such as "Is windshield damage covered under this motor policy, and what is the deductible?" or "Which documents are required for a travel medical claim above 5,000?" The answers live in long PDFs with definitions sections, exclusion tables, endorsements that override earlier clauses, and cross-references. Reading takes minutes per question. Generic chatbots invent coverage. The system must be right, show where the answer comes from, or say clearly that the documents do not answer the question.

4. Users and use cases

Personas

- Policy servicing agent: needs fast, cited lookups while on a call.
- Claims handler: checks coverage, exclusions and required documents. Project 2 calls this system as a tool.
- Compliance analyst: compares wording across products or insurers.

Use cases

ID	Use case	Example question
UC-1	Coverage lookup	"Does the home policy cover accidental damage to laptops away from home?"
UC-2	Limits and deductibles from tables	"What is the per-claim deductible for windshield replacement?"
UC-3	Definition lookup	"How does the policy define 'pre-existing condition'?"
UC-4	Required documentation	"What must be submitted for a baggage delay claim?"
UC-5	Multi-hop across sections	"Is flood covered if the optional endorsement was purchased?"
UC-6	Out of scope or unanswerable	"What is the premium for a 30-year-old?" (not in wording)

5. Goals and non-goals

Goals

1. Grounded answers with page-level citations that a reader can verify.
2. Explicit refusal when the evidence is insufficient, with a pointer to what was found.
3. Published quality: retrieval metrics, answer faithfulness, refusal accuracy, latency and cost, per pipeline configuration.
4. Production shape: streaming API, tracing, tests, Docker, one-command local setup, deployed demo, CI that runs evals.
5. A clean internal API that Projects 2 and 4 can call.

Non-goals for version 1

- Multi-turn conversation memory. Single-turn Q&A; conversation-aware rewriting is a stretch goal.
- Fine-tuning any model.
- Knowledge graphs or GraphRAG.
- Multi-tenant authentication or billing.
- An ingestion UI. Ingestion is a CLI.
- OCR tuning beyond Docling defaults.

6. Success metrics

Metric	Target	How it is measured
Hit@5 (gold page in top 5 retrieved)	at least 0.90	Golden set, programmatic
MRR (mean reciprocal rank of first gold page)	at least 0.75	Golden set, programmatic
Faithfulness (answer supported by retrieved context)	at least 0.90	LLM judge calibrated against 30 hand labels
Correct refusal rate on unanswerable questions	at least 0.90	Golden set, programmatic
False refusal rate on answerable questions	at most 0.05	Golden set, programmatic
Citation accuracy (cited page contains the quoted span)	at least 0.95	Programmatic string match after normalisation
Time to first token	at most 1.5 seconds	Langfuse traces, p50
End-to-end latency	p95 at most 8 seconds	Langfuse traces
Cost per query	at most 0.03 USD with the flagship model	Token accounting per trace
Ablation	Each stage shows a measured delta	Four configurations, three runs each

7. Functional requirements

FR-1 Ingestion. A CLI ingests a folder of PDFs with Docling, preserving page numbers, section headings and tables (tables rendered as Markdown). Ingestion is idempotent: re-running on the same file does not duplicate chunks. A manifest records document id, insurer, product, document type, effective date, page count and content hash.

FR-2 Chunking. Structure-aware chunking that never splits a table row or a numbered clause mid-sentence, target 300 to 800 tokens with modest overlap. Each chunk carries a contextual header (document title, section path) prepended to the text used for embedding and retrieval. Parent-child chunking (small chunk retrieved, larger parent returned) is optional.

FR-3 Indexing. Postgres with pgvector (HNSW index on embeddings) and a full-text search column for lexical retrieval. Metadata columns support filtering by insurer, product, document type and effective date. Index versioning allows re-embedding without downtime.

FR-4 Query understanding. A cheap-tier model rewrites the user question: spelling normalisation, expansion of insurance abbreviations, decomposition of multi-part questions into sub-queries, and detection of clearly out-of-scope questions.

FR-5 Hybrid retrieval. Lexical top 50 and dense top 50 fused with reciprocal rank fusion into a top 20 candidate list. Filters apply before fusion.

FR-6 Reranking. A cross-encoder reranker (hosted or open) reorders the top 20 into the final top 5 to 8 passages. A relevance threshold decides whether any evidence is strong enough to answer.

FR-7 Answer generation. A flagship-tier model produces a structured output: answer text, list of citations (document id, page, exact quote), a confidence value, and an `insufficient_evidence` flag. The prompt requires that every factual claim be supported by a quoted span from the retrieved context and forbids using outside knowledge about insurance.

FR-8 Refusal. If no passage passes the relevance threshold, or the model sets `insufficient_evidence`, or the post-generation citation check fails, the system returns an explicit refusal that names the closest sections found and suggests a rephrasing.

FR-9 Streaming API. `POST /ask` streams the answer over Server-Sent Events, first tokens within 1.5 seconds, with a final event carrying citations, confidence and trace id. A non-streaming variant returns the same structure as JSON. A `POST /retrieve` endpoint exposes retrieval alone for Projects 2 and 4.

FR-10 Citation verification. After generation, every citation's quote is checked against the stored page text. Unverifiable citations are removed and, if none remain, the answer is converted to a refusal.

FR-11 User interface. A minimal web UI: question box, streaming answer, clickable citations that open the cited page text with the quote highlighted, and a visible "insufficient evidence" state.

FR-12 Evaluation CLI. `eval run --config <name>` runs the golden set through a named configuration and writes metrics as JSON plus a Markdown table. `eval ablation` runs all configurations three times and produces the ablation table with mean and standard deviation.

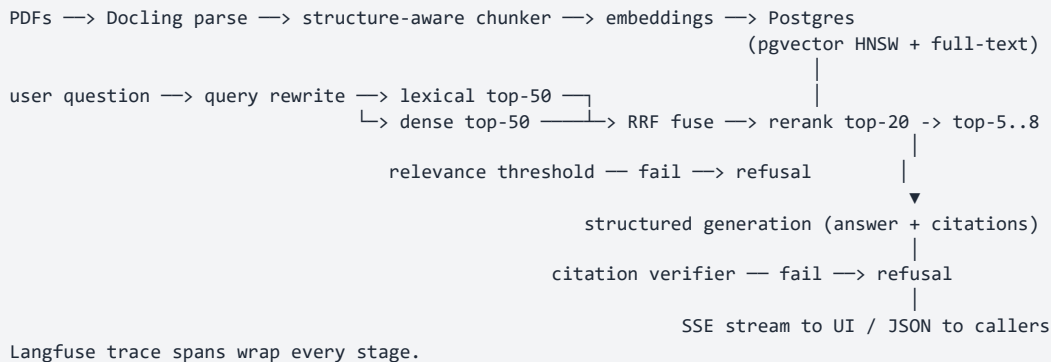
FR-13 Observability. One Langfuse trace per question with spans for rewrite, lexical retrieval, dense retrieval, fusion, rerank, generation and verification, each with latency, token counts and cost. Evaluation scores attach to traces.

FR-14 Admin. CLI commands to re-index, show corpus statistics and export the golden set.

8. Non-functional requirements

- **Reliability.** Provider timeouts and retries with backoff; fallback to a second model provider for generation; graceful degradation to lexical-only retrieval if the embedding provider is down.
- **Reproducibility.** Pinned dependency versions, seeded evaluation order, configuration captured in every eval report (model ids, prompt version, retriever settings, git commit).
- **Testing.** Unit tests for chunker, fusion, citation verifier and refusal logic; integration test for ingest-retrieve-answer on a three-page fixture; the evaluation smoke subset (10 questions) runs in CI on every pull request.
- **Packaging.** Docker Compose brings up API, Postgres with pgvector and the UI with one command. A Makefile or task runner wraps ingest, eval and test.
- **Deployment.** One cloud target with managed Postgres (for example AWS App Runner or ECS with RDS). Secrets from environment variables only.
- **Security.** API key on the public endpoint; rate limiting per key; no personal data in the corpus.
- **Cost control.** Cheap-tier model for rewriting and judging during development; prompt caching for the static system prompt and few-shot examples; batch API for full eval runs.

9. Architecture



Components: `ingest` (CLI), `index` (Postgres schema and migrations), `retrieval` (rewrite, hybrid, fusion, rerank), `answer` (generation, verification, refusal), `api` (FastAPI), `ui` (static web app), `evals` (golden set, runners, reports).

10. Technology decisions

Decision	Chosen	Alternatives considered	Why
Vector store	Postgres with pgvector	Qdrant, Pinecone, Weaviate, Chroma	One database for vectors, lexical search and metadata; the most common default in postings and teams; teaches SQL. Qdrant is the recommended alternative if multi-vector or very large scale is needed.
Parsing	Docling	PyMuPDF, Unstructured, LlamaParse	Best open-source table and layout fidelity; on Thoughtworks Radar Trial; no vendor lock-in.
Lexical retrieval	Postgres full-text search	Elasticsearch, OpenSearch, rank_bm25 in memory	Zero extra infrastructure; good enough at this corpus size; document the BM25 difference.
Fusion	Reciprocal rank fusion	Weighted score fusion	Robust without score calibration.
Reranker	Hosted cross-encoder (Cohere Rerank) with an open fallback (bge-reranker)	No reranker, LLM-based reranking	Largest quality gain per unit of complexity; the ablation proves it.
Generation model	Current flagship tier with a cheap-tier fallback	Single model	Provider-agnostic design is expected; fallback demonstrates resilience.
Embeddings	One hosted embedding model, version pinned	Open model via local inference	Simplicity first; note the swap path in DECISIONS.md.
API framework	FastAPI	Flask, Django, Litestar	Async, typed, SSE support, the de facto standard in postings.

Decision	Chosen	Alternatives considered	Why
Observability	Langfuse	LangSmith, Arize Phoenix	Open source, dominant in downloads, datasets and scores API.
Orchestration	Plain Python	LangChain, LlamaIndex	The pipeline is a dozen functions; a framework would hide the prompts and the decisions.

11. Data plan

Corpus. Five to ten publicly available policy wordings from at least three insurers across two product lines (for example motor and travel), plus two or three regulator guidance documents. Target 300 to 800 pages total. Selection criteria: real tables (limits, deductibles), definitions sections, endorsements, at least one scanned or poorly formatted document.

Golden set. 80 to 100 questions stored as versioned JSONL with fields: id, question, category, expected answer, gold document ids and pages, difficulty, notes.

Category	Count	Purpose
Answerable, single passage	35	Core retrieval and grounding
Answerable, table lookup	15	Table parsing and numeric fidelity
Answerable, multi-hop	10	Exclusion plus endorsement reasoning
Partially answerable or ambiguous	15	Calibration of confidence and hedging
Unanswerable but plausible	20	Refusal behaviour

Construction. Draft candidates with a model from each document section, then verify every question, answer and gold page by hand. Write a one-page labelling guideline: what counts as answered, how to mark a partial answer, when refusal is the correct outcome. Hold out 20 percent of questions and never tune on them.

12. Evaluation plan

1. **Retrieval metrics.** Hit@5, Hit@10 and MRR against gold pages for every configuration.
2. **Answer quality.** Faithfulness and answer correctness scored by an LLM judge with a written rubric; the judge is calibrated against 30 hand-labelled answers and its agreement rate is published.
3. **Refusal metrics.** Correct refusals on the unanswerable subset; false refusals on the answerable subset.
4. **Citation accuracy.** Programmatic verification that each quoted span exists on the cited page.
5. **Ablation.** Configurations A: dense top 5 only; B: hybrid with RRF; C: B plus reranker; D: C plus query rewriting. Three runs each, report mean and standard deviation, latency and cost per configuration.
6. **Error analysis.** For the best configuration, read every failure, code it into a failure taxonomy (parsing, chunk boundary, retrieval miss, rerank miss, generation unfaithful, wrong refusal), and publish the top five categories with counts and one example each.

13. Failure modes and mitigations

Failure mode	Mitigation
Table parsed into unreadable text	Docling table mode; table-aware chunker; a table-specific subset in the golden set
Chunk boundary splits a definition or clause	Structure-aware splitting on headings and numbered clauses; overlap; parent-child fallback
Retriever returns the wrong insurer's document	Metadata filters from the question or UI context; insurer name in chunk headers
Reranker adds latency	Rerank only the top 20; measure and publish the latency cost
Model answers from prior knowledge	Quote-grounded prompt; citation verifier; judge rubric penalises unsupported claims
Hallucinated citation	Post-generation verification removes unverifiable citations and downgrades to refusal
Context too long for the model	Cap at top 8 passages; truncate by section, never mid-table
Embedding model version changes	Index versioning; re-embed offline; switch atomically
Provider outage	Timeouts, retries, second provider for generation, lexical-only degradation for retrieval
Golden set overfitting	20 percent hold-out; never tune prompts on hold-out questions

14. Security and safety

- Document text is untrusted input. The prompt places retrieved passages inside clearly delimited data blocks, and the golden set includes one deliberately poisoned document with embedded instructions to verify they are ignored.
- No personal data in the corpus. Policy wordings are public templates.
- Public endpoint requires an API key and enforces per-key rate limits.
- Every answer carries a visible disclaimer that it is an extract from policy documents, not advice.

15. Build plan

Week	Deliverables	Done when
3	Corpus chosen and stored; Docling ingestion CLI; Postgres schema; chunker with tests	Re-ingest is idempotent; tables preserved; page numbers correct on a sample
4	Embeddings; lexical and dense retrieval; RRF; golden set v1 with 50 questions; eval CLI computing Hit@k and MRR	First retrieval numbers recorded in EVALS.md
5	Query rewriting; reranker; structured generation; citation verifier; refusal logic; SSE API	Citation accuracy at or above 0.95 on the golden set
6	Golden set to 80 to 100; LLM judge with calibration; ablation runner; Langfuse tracing; CI smoke eval	Ablation table complete with three runs per configuration

Week	Deliverables	Done when
7	Web UI; Docker Compose; cloud deploy; README with architecture; DECISIONS.md; EVALS.md; write-up 1	Live demo URL; blog post published; repository pinned

16. Deliverables checklist

- Repository with README (problem, architecture diagram, quick start, demo link), DECISIONS.md (every row of section 10 plus what changed during the build), EVALS.md (metrics, ablation table, error taxonomy, how to reproduce).
- Live demo URL and a two-minute video showing a correct cited answer, a refusal, and the trace in Langfuse.
- Golden set and labelling guideline committed.
- CI running lint, tests and the smoke eval on every pull request.
- Write-up 1: "What hybrid retrieval and reranking actually bought me, with numbers."

17. Resume bullets (fill with real numbers)

- Built a grounded Q&A system over [N] pages of insurance policy documents using structure-aware parsing, hybrid lexical and dense retrieval with reciprocal rank fusion, and cross-encoder reranking; reranking lifted Hit@5 from [X] to [Y] and MRR from [X] to [Y] against a naive dense baseline.
- Reached [X] percent faithfulness and [Y] percent correct refusal on unanswerable questions across a [N]-question golden set, scored by an LLM judge calibrated to [Z] percent agreement with hand labels; evals run in CI on every pull request.
- Streams first tokens in [X] seconds with p95 end-to-end latency of [Y] seconds at [Z] USD per query; deployed on [cloud] with Docker and traced end-to-end in Langfuse.

18. Interview questions this project prepares you for

- Design an enterprise document Q&A system. Where does it fail and how do you know?
- How do you decide whether a bad answer is a retrieval problem or a generation problem?
- How would you chunk a 200-page PDF with tables? When do you add a reranker?
- How do you stop the model answering from its own knowledge?
- Long context versus retrieval: when is each right?
- How do you build a golden set, and how do you keep from overfitting to it?
- How do you calibrate an LLM judge?

19. Stretch goals

- Conversation-aware query rewriting for follow-up questions.
- Multilingual corpus and cross-lingual retrieval.
- Contextual retrieval (model-generated chunk context) compared against plain contextual headers in the ablation.
- Cross-insurer comparison mode that answers "how do policies A and B differ on X".
- Exposure through the Project 4 MCP server.

20. Open questions

- Which two product lines give the richest tables and exclusions? Decide in week 3 after skimming candidate documents.
- Hosted versus open reranker as the default: decide on measured latency and cost in week 5.